

PERSONAL PROGRAMMING PROJECT

TOPIC:

Parameter identification of an elastic-plastic material using an axisymmetric FEM-program

Supervisor: Dr.-Ing. Martin Abendroth

Name	Matriculation number
Ghogul, Gopal Kalpana ghogul.gopal-kalpana@student.tu-freiberg.de	63942

Contents

1. Introduction :	3
2. Theory :	3
2.1 Main program:	3
2.2 FEM model:	5
3. Numerical methods and implementation:	7
3.1 Levenberg-Marquardt algorithm:	7
3.2 FEM model:	9
4. Results of tests and discussion:	12
Test cases:	17
5. Manual:	22
6. Literature:	24
7. Git log:	25

1. Introduction :

Parameter identification is a mathematical concept widely used to determine the optimal parameters of a complex material model Eg: non-linear (elastic-plastic, viscoelastic). In this problem, I coupled algorithm and FEM simulation to optimize the parameters which is a challenging task to do as a program. Other than that, Parameter identification is used in material modeling and machine learning and in various fields. These include maximizing factors such as productivity, reliability and efficiency. So, these specs motivate me to do this project in parameter identification.

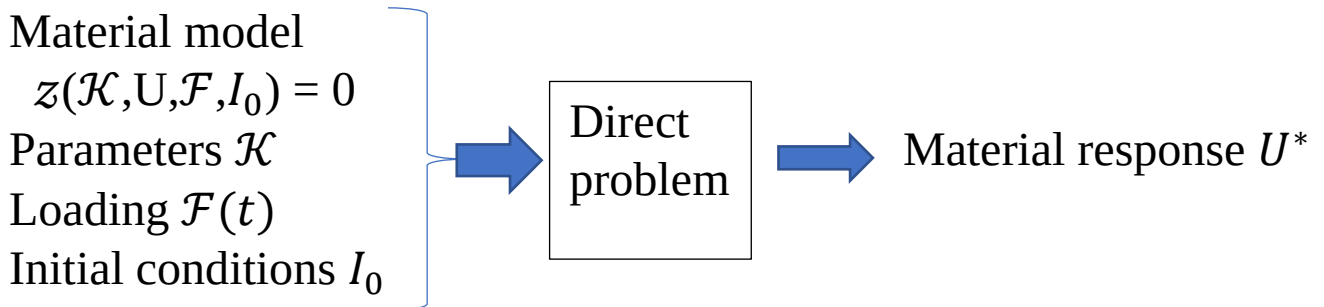
2. Theory :

2.1 Main program:

As I mentioned, parameter identification is an estimation strategy to find unknown parameters of a material model. There are two ways to solve the problem, direct method and indirect(inverse) method.

Direct method:

The direct problem determines the material response of the given model. Material parameters, loading history and initial conditions are inputs of the problem.



Indirect method:

The indirect problem determines the material parameters of the given model. One searches for the optimal set of parameters for the given material model,

loading history, initial conditions by comparing the computed data and measured data.

Material model

$$z(\mathcal{K}, \mathbf{U}, \mathcal{F}, I_0) = 0$$

Loading $\mathcal{F}(t)$

Experimental data σ, ε

Initial condition I_0

Indirect
problem

Optimal parameters $\mathcal{K}^*$

From the above methods, I am using the indirect method to find the material parameters. In the indirect method, the unique solution typically does not exist. We have to seek for the optimal fit between measured data and simulation data in the sense of least square methods.

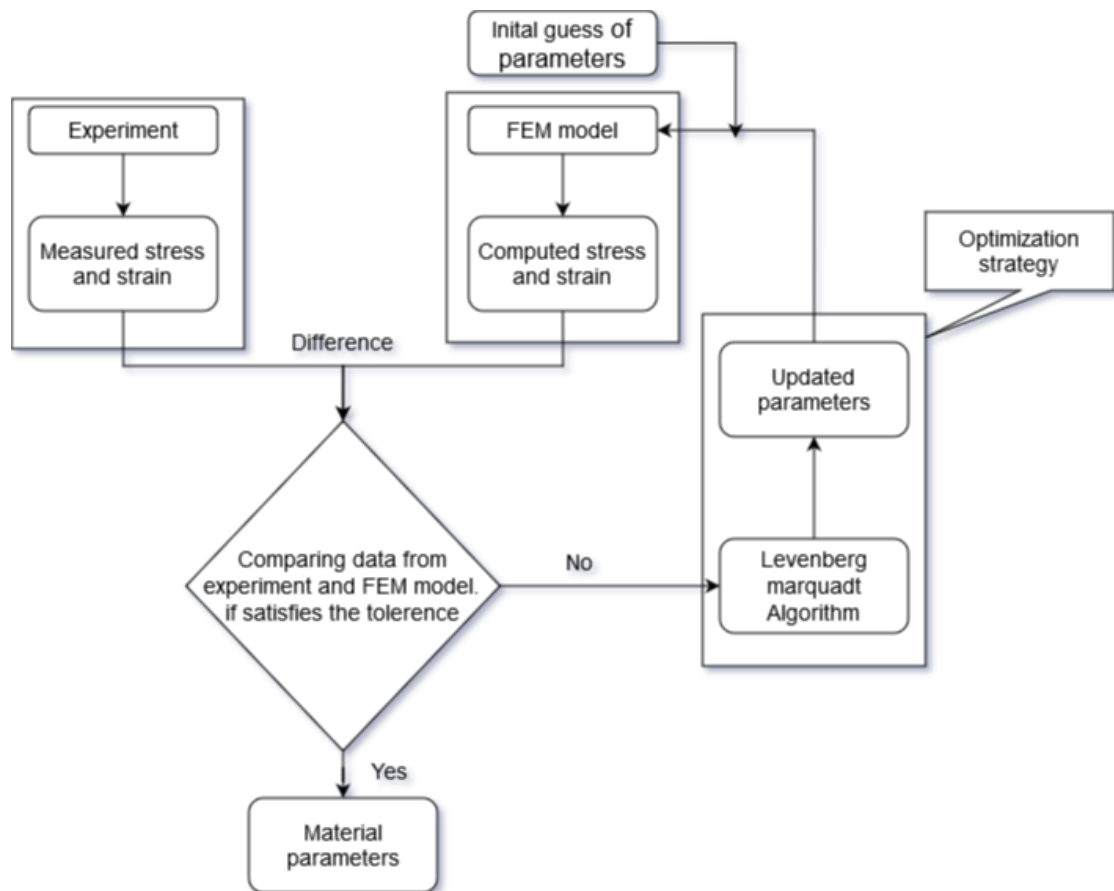


Fig: 1 Main program

Firstly, simulation has to be done for the initial guess and compare the results with experimental data. If it is not converged then the Levenberg-Marquardt algorithm will come in to play, it will iterative over and over until the errors are minimized and the optimal parameter will get as an output. Levenberg-Marquardt algorithm call Fem program for each iteration until the errors are minimized. It seems simple but implementation as a program is quite a challenging task.

2.2 FEM model:

In FEM, the typical tensile test has to be done, for that axisymmetric model is considered. Axisymmetric elements are symmetric in geometry and loading about an axis. It is customary to align this symmetry axis with the z-axis of the cylindrical (r, θ, z) coordinate system. r-radial direction, θ -circumferential direction, and z- longitudinal direction.

Boundary conditions, loading conditions and material properties are symmetric about z-axis so, they are all independent of θ -circumferential direction.

The simulation data is a stress-strain curve so, FEM has to model for elastic-plastic behavior, for elasticity general Hook's law and for plasticity Von-mises plasticity with non-linear isotropic hardening is used.

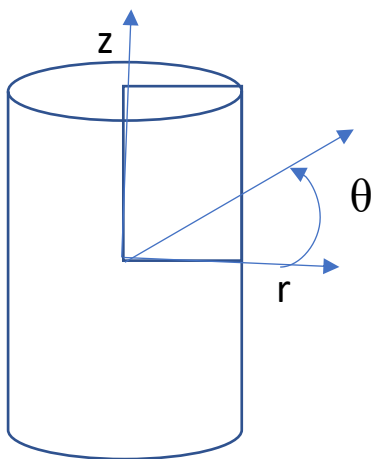


Fig: 2 Axisymmetric model

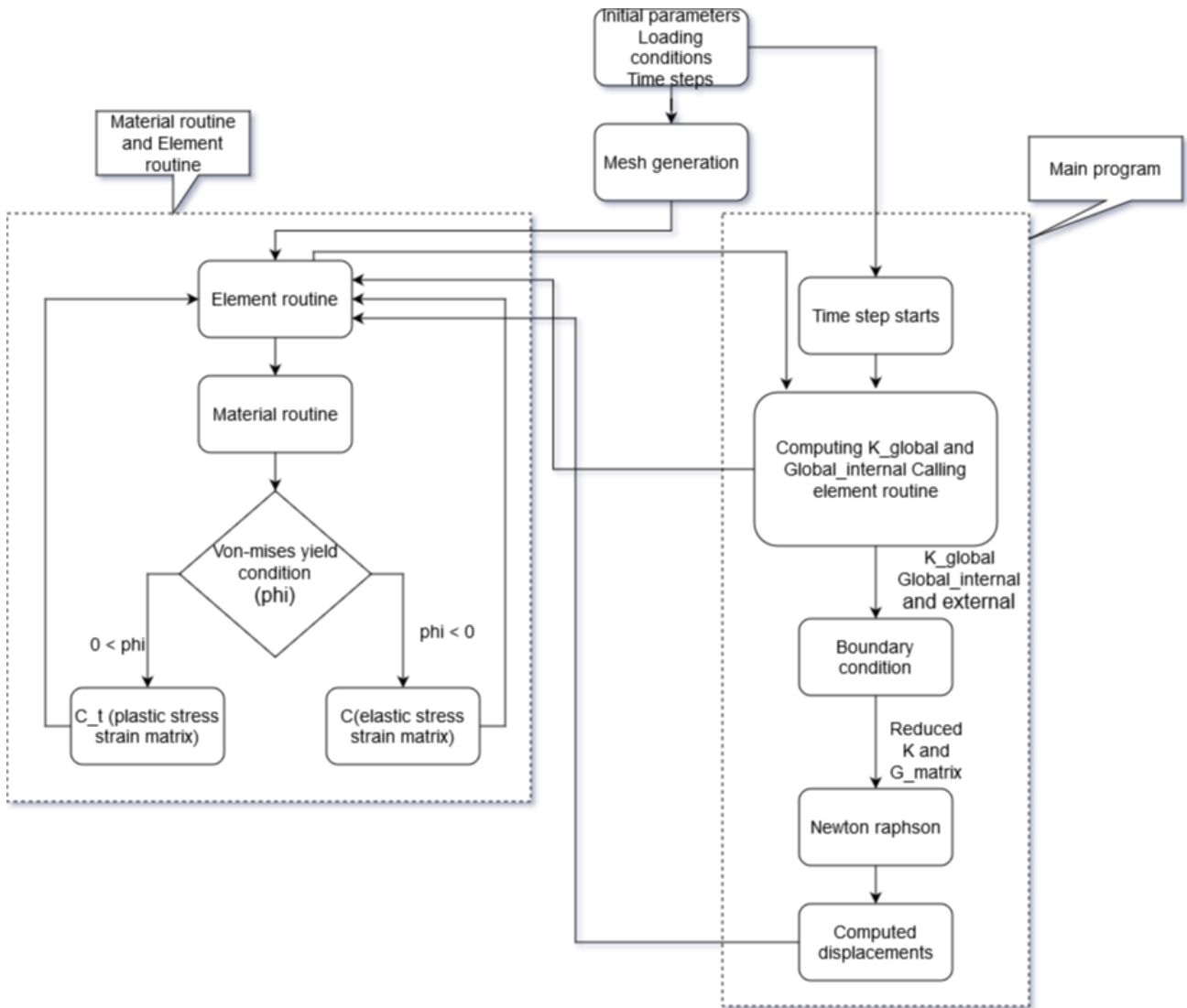


Fig: 3 FEM flowchart

3. Numerical methods and implementation:

3.1 Levenberg-Marquardt algorithm:

Parameters can be identified by implementing various algorithms, in that gradient-free method, gradient-based method(line search method), trust region method, non-linear least square method are kinds of optimization methods. In this specific problem, I am using the non-linear least square method, this method is comparatively easier to implement and computationally cheaper.

Levenberg-Marquardt algorithm is widely used for finding the parameters. Before digging deep into the Levenberg-Marquardt algorithm, one has to understand the trust-region algorithm principle. The Trust region is the subset region of the objective function(FEM model). If the objective function found within the trust region then the region is expanded, if the approximation is poor then the region will contract. The fit is evaluated by the expected data from the model with actual data observed in the objective function. The Trust region uses step size and then step direction.

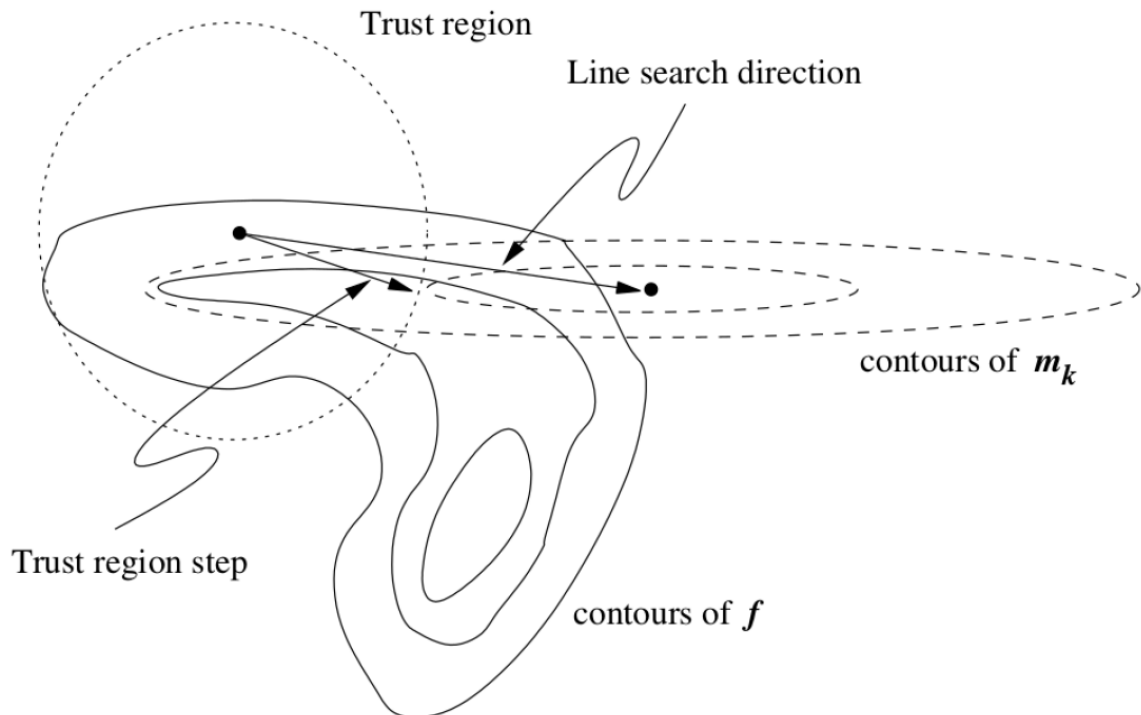


Fig: 4 Trust region source: literature 3

Pseudocode Levenberg-Marquardt algorithm:

Initialize $k_0 = [k_1 \dots k_n]$, $\lambda = 0.001$, f_{tol} , $\eta = 0.25$, i_{max} , j_{max} , $i = 1$, $j = 1$

Set $r_0 = residual(k_0)$

Set $f_0 = \frac{1}{2} \|r_0\|^2$

Set $j_0 = jacobian(k_0)$

Set $m_0 = f_0$

While $\Delta f > f_{tol}$ and $i < i_{max}$ and $j < j_{max}$:

Set $\Delta k = lstsq([J_{i-1}\sqrt{\lambda}I]^T, -[r_{i-1}0]^T)$

Set $m_i = f_{i-1} + \Delta k^T j_{i-1} + \frac{1}{2} \Delta k^T j_{i-1}^T j_{i-1} \Delta k$

Set $k_i = k_{i-1} + \Delta k$

Set $r_i = residual(k_i)$

Set $f_i = \frac{1}{2} \|r_i\|^2$

Set $j_i = jacobian(k_i)$

Set $\Delta f = |f_i - f_{i-1}|$

Set $\rho_i = \frac{f_{i-1} - f_i}{m_{i-1} - m_i}$

If $\rho_i \leq \frac{1}{4}$:

Set $\lambda_i = 10\lambda_{i-1}$

Else:

if $\rho_i \geq \frac{3}{4}$:

Set $\lambda_i = 0.1\lambda_{i-1}$

Else

Set $\lambda_i = \lambda_{i-1}$

If $\rho_i > \eta$:

accept step

set $j = 1$

set $i = i + 1$

Else:

reject step

set $k_i = k_{i-1}$

set $j = j + 1$

end while

Levenberg-Marquardt can be described using a trust-region framework. In pseudocode, m_i defines the trust-region radius, if the objective function in the trust-region radius, then λ_i is increased or it will decrease. If ρ_i is greater than the η , step is accepted and it will try to optimize the parameters, if the step is rejected, then trust-region radius only changes the initial guess will not optimize. The trick is to change the trust-region size (λ). At each iteration, the damped quadratic fit predicts a certain reduction in the cost function, which we would expect to be a smaller reduction than the true reduction. If the guess is poor then it will continuously change the trust-region and direction in each iteration and it may never converge.

3.2 FEM model:

Tensile test simulation is needed to solve the optimization problem. The circular bar is considered for the simulation. To solve this test, the axisymmetric model is considered.

The general strong form :

$$0 = \rho \ddot{u}_j - \sigma_{ij,i} - f_j$$

The weak form is by multiply by an arbitrary test function in the strong form, which yields.

$$\begin{aligned} \delta w &= \int_{\Omega} (\rho \ddot{u}_j - \sigma_{ij,i} - f_j) \delta u_j dv = 0 \\ \delta w &= \underbrace{\int_{\Omega} \sigma_{ij,i} \delta \varepsilon_{ij} dv}_{\text{internal}} + \underbrace{\int_{\Omega} \rho \ddot{u}_j \delta u dv}_{\text{kinetics}} - \underbrace{\left[\int_{\Omega} f_j \delta u_j dv + \int_{\Omega} t_j \delta u_j dA \right]}_{\text{external}} = 0 \end{aligned}$$

In axisymmetric case,

$$\sigma = \begin{bmatrix} \sigma_{rr} \\ \sigma_{zz} \\ \sigma_{\theta\theta} \\ \tau_{rz} \end{bmatrix} \quad \varepsilon = \begin{bmatrix} \varepsilon_{rr} \\ \varepsilon_{zz} \\ \varepsilon_{\theta\theta} \\ \gamma_{rz} \end{bmatrix} \quad U = \begin{bmatrix} u(r, z) \\ v(r, z) \end{bmatrix}$$

Stress-strain relationship matrix

$$C = \frac{E}{(1+\nu)(1-2\nu)} \begin{bmatrix} (1-\nu) & \nu & \nu & 0 \\ \nu & (1-\nu) & \nu & 0 \\ \nu & \nu & (1-\nu) & 0 \\ 0 & 0 & 0 & \frac{(1-2\nu)}{2} \end{bmatrix}$$

We know that $\epsilon = B.U$, $B = \frac{\partial[N]}{\partial X} = J^{-1} \frac{\partial[N]}{\partial \xi}$, $J = \frac{\partial[N]}{\partial \xi} X_e$, X_e = physical domain

$$\epsilon = \begin{bmatrix} \frac{\partial u}{\partial r} \\ \frac{\partial v}{\partial r} \\ \frac{u}{r} \\ \frac{\partial u}{\partial r} + \frac{\partial v}{\partial r} \end{bmatrix}$$

Isoparametric 2D-rectangular element with bi-linear shape function is considered.

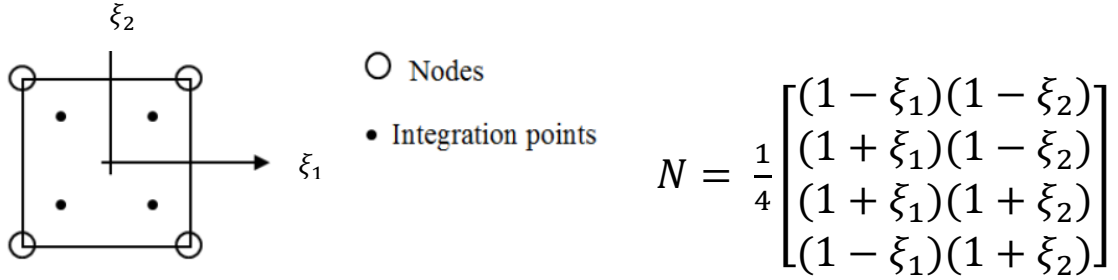


Fig: 5 rectangular element

From the weak form, we can derive element stiffness matrix $[k]$, internal(f_{int}) and an external force matrix(f_{ext}), body force not considered and this is a static problem so, kinetics is also not considered.

$$f_{int} = \int_{\Omega} \sigma_{ij,i} \delta \epsilon_{ij} dv$$

For circumferential differential element integral becomes

$$f_{int} = 2\pi \int_A \sigma_{ij,i} \delta \varepsilon_{ij} \, dr \, dz$$

$$f_{int} = 2\pi \int_A \sigma_{ij,i} B^T \, r \, dr \, dz$$

$$[k] = 2\pi \int_A B^T C B \, r \, dr \, dz$$

$$f_{ext} = 2\pi \int_s N^T f \, r \, ds$$

In the numerical integration, the gauss quadrature method is used. In our case the integration order is 2 so, two gauss point has to be employed. For visualization please see fig: 5. Polynomial degree of the integrand is 2(p-1), for bi-linear shape function p=2.

$$w_\alpha = 1 \text{ and } \xi_\alpha = \pm \sqrt{\frac{1}{3}}$$

$$f_{int} = w \cdot w \cdot 2\pi r \cdot \sigma \cdot B^T \cdot \|J\|$$

$$[k] = w \cdot w \cdot 2\pi r \cdot B^T \cdot C \cdot B \cdot \|J\|$$

$$f_{ext} = w \cdot w \cdot 2\pi r \cdot N^T \cdot f \cdot \|J\|$$

Von-mises plasticity with isotropic hardening is used to determine the plastic behavior of the material. To check yield criteria for von-mises plasticity.

$$\phi = \sigma_{eq} - \sigma_y(\varepsilon^{pl})$$

$$\sigma_{eq} = \sqrt{\frac{3}{2} \sigma_{dev} : \sigma_{dev}}$$

If $\phi > 0$ plasticity is initiated. Then have to calculate the plastic multiplier. Ensures radial return mapping (plastic corrector).

$$\gamma_{n+1} = \frac{\phi}{2\mu + \frac{2}{3}h}$$

Then the current stress and strain have to be calculated.

$$\begin{aligned}\varepsilon^{pl} &= \varepsilon_n^{pl} + \gamma_{n+1} \mathbf{n}_{n+1} \\ dev(\sigma_{n+1}) &= dev(\sigma_{n+1}^{trial}) - 2\mu\gamma_{n+1} \mathbf{n}_{n+1} \\ \sigma_{n+1} &= dev(\sigma_{n+1}) + k \, tr(\varepsilon_{n+1}) I\end{aligned}$$

The algorithmic tangent stiffness matrix can be calculated by $\mathbf{C}_{n+1} = \frac{d\sigma_{n+1}}{d\varepsilon_{n+1}}$.

$$\begin{aligned}\mathbf{C}_{n+1}^{dev} &= 2\mu\beta_1 \mathbb{P}_{sym} - 2\mu\beta_2 \mathbf{n}_{n+1} \otimes \mathbf{n}_{n+1} \\ \mathbf{C}_{n+1} &= \mathbf{C}_{n+1}^{dev} + k I \otimes I\end{aligned}$$

Where $\beta_1 = 1 - \frac{\phi_n}{\|\varepsilon_{n+1}\|} \frac{1}{1 + \frac{h}{3\mu}}$

$$\beta_2 = \left[1 - \frac{\phi_n}{\|\varepsilon_{n+1}\|}\right] \frac{1}{1 + \frac{h}{3\mu}}$$

3.3 Programming language and libraries:

To implement the above problem python programming language with NumPy and matplotlib libraries are used.

4. Results of tests and discussion:

Briefly, my task is to find the material parameters such as yield stress, elasticity modulus, hardening modulus, hardening exponents. Here I programmed a script that will optimize the parameters based on the given experimental data.

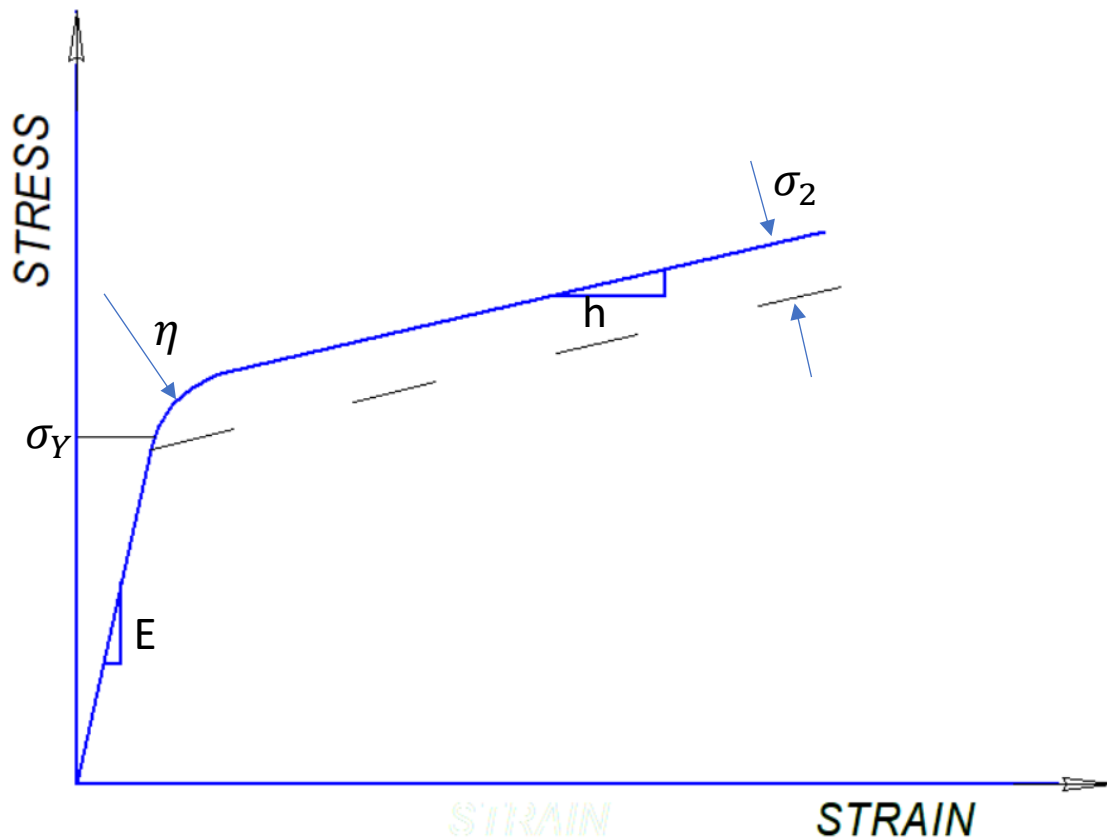


Fig: 6 stress-strain curve

The parameters we are going to find from the program are yield stress(σ_y), young's modulus(E), linear hardening modulus(h), nonlinear hardening modulus(σ_2), hardening exponent(η). See fig: 6. Firstly, the user has to input the parameters as initial guess and output will be the optimized parameters for the provided experimental data.

Results:

Parameters	Input (MPa)	Output(MPa)
Young's modulus(E)	200000	212742.99
Yield stress(σ_y)	600	501.338
Linear hardening modulus(h)	300	241.325
Nonlinear hardening modulus(σ_2)	300	251.029
Hardening exponent(η)	18	16.3

Table: 1

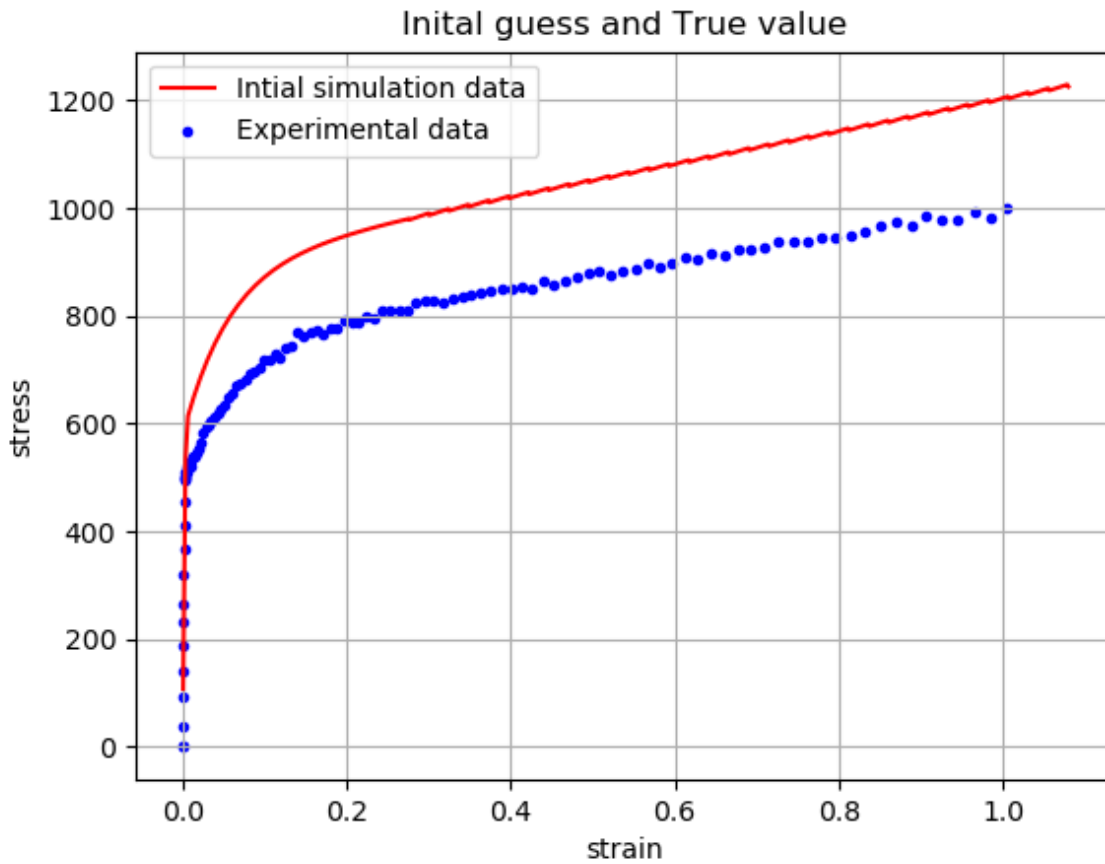


Fig: 7 initial guess and experimental data

The above plot shows the difference between the initial guess and the experimental data. So, the Levenberg-Marquardt algorithm is used to fit the curve between the experimental data points and finding the optimal parameters of the experimental data.

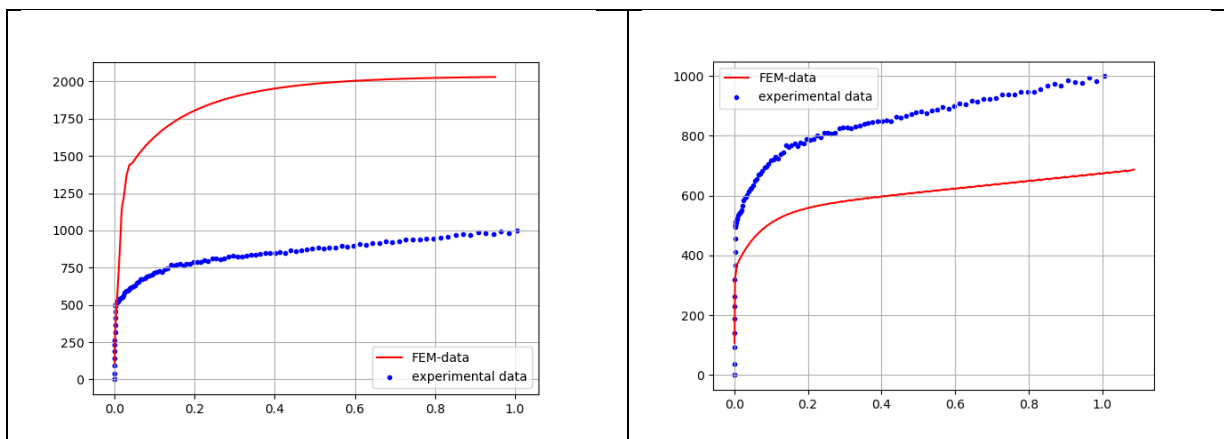


Fig: 8a

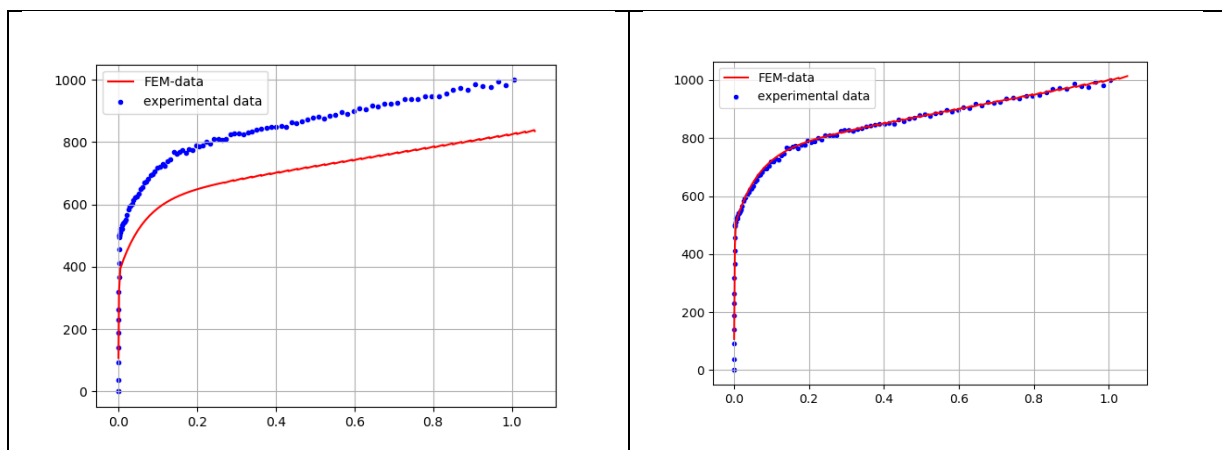


Fig: 8a & 8b Plots from random iteration steps

The above plots depict how the algorithm optimized the parameters over the iterations.



Fig: 9 simulation data and training data

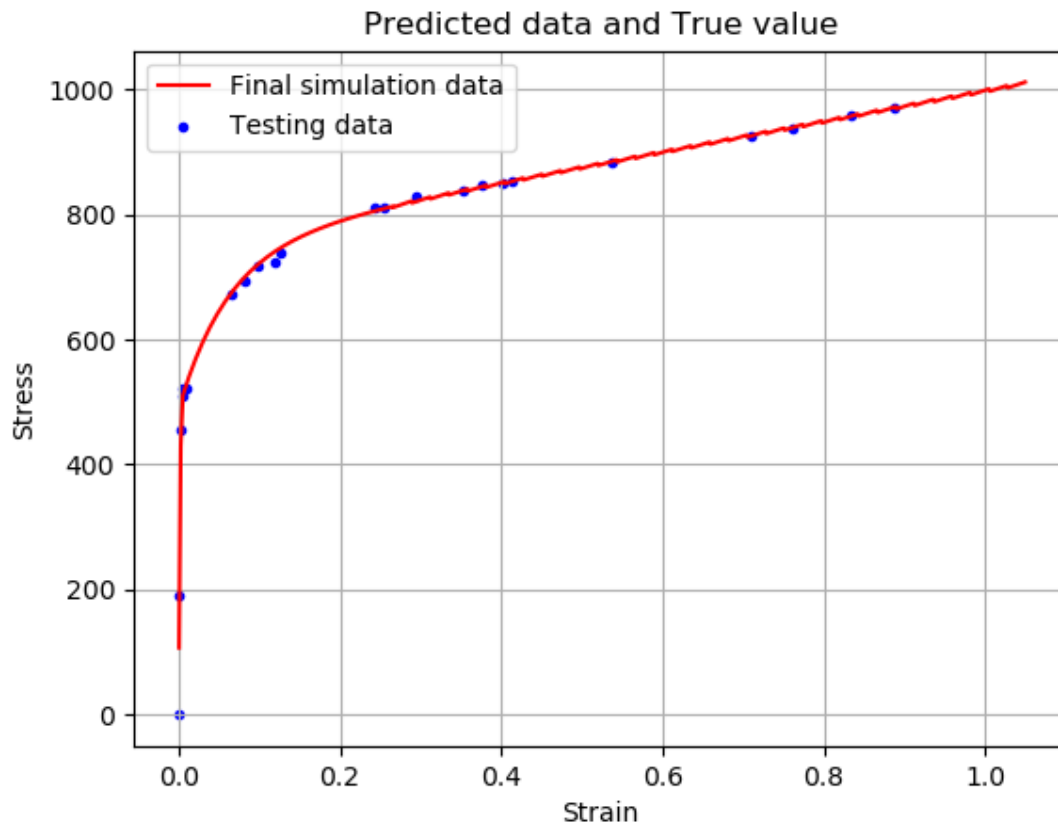


Fig: 10 Testing and simulation data

Experimental data are split into training data and testing data. So for optimization, only training datasets are used. And testing data are used to check the optimized result.

In fig: 9 the graph shows that the simulation data fit with training data. And in fig: 10 the graph shows that the testing data fits with simulation data.

Remark:

The initial guess of the parameters should be a good guess, which means the guess should not deviate largely from the original parameters. Before any proceedings, I would suggest taking a look at the experimental data.

Mean squared error is the average of the square of the errors. That is the average squared difference between the simulation data and experimental data. The below graph shows how the errors are reduced over the iterations.

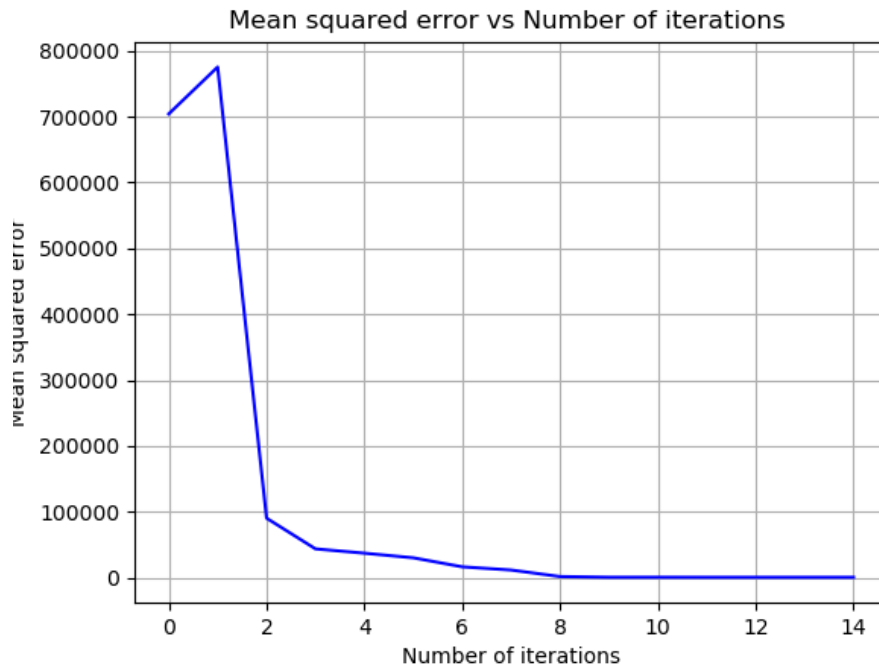


Fig: 10 Mean squared error

Test cases:

Testing is verifying the program input with the known output. Here I have done some testing for the algorithm and FEM-model.

Test case: 1

Algorithm testing, I have taken a polynomial function with known coefficients and checking with the algorithm output whether the algorithm is optimizing the data and gives nearly the same coefficients as output.

True Polynomial function = $2x^3 + 5x^2 + 8x$

Objective function = $ax^3 + bx^2 + cx$

Coefficients	Input	Output	Expected output
a	10	1.88913	2
b	20	5.40413	5
c	25	7.64956	8

Table: 2

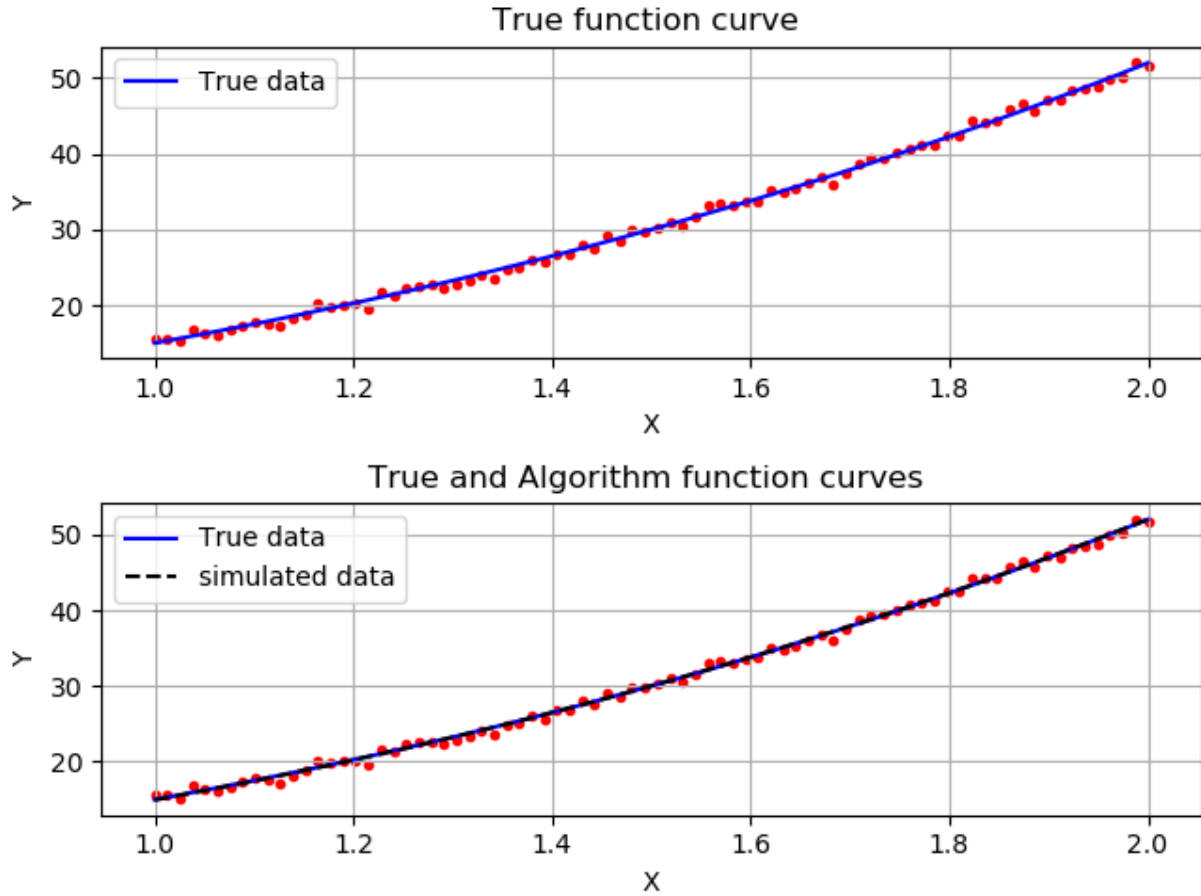


Fig: 11 True and objective function fits

Test case: 2

FEM testing, my main purpose of this project is to find the parameters of the material based on the stress-strain curve. I got the stress-strain curve from my supervisor and checking whether my FEM program generates the same curve or not for the specific material parameters. The material parameters given below is the typical steel of grade B500.

Parameters	Input (MPa)
Young's modulus(E)	210000
Yield stress(σ_y)	500
Linear hardening modulus(h)	250
Nonlinear hardening modulus(σ_2)	250
Hardening exponent(η)	15

Table: 3

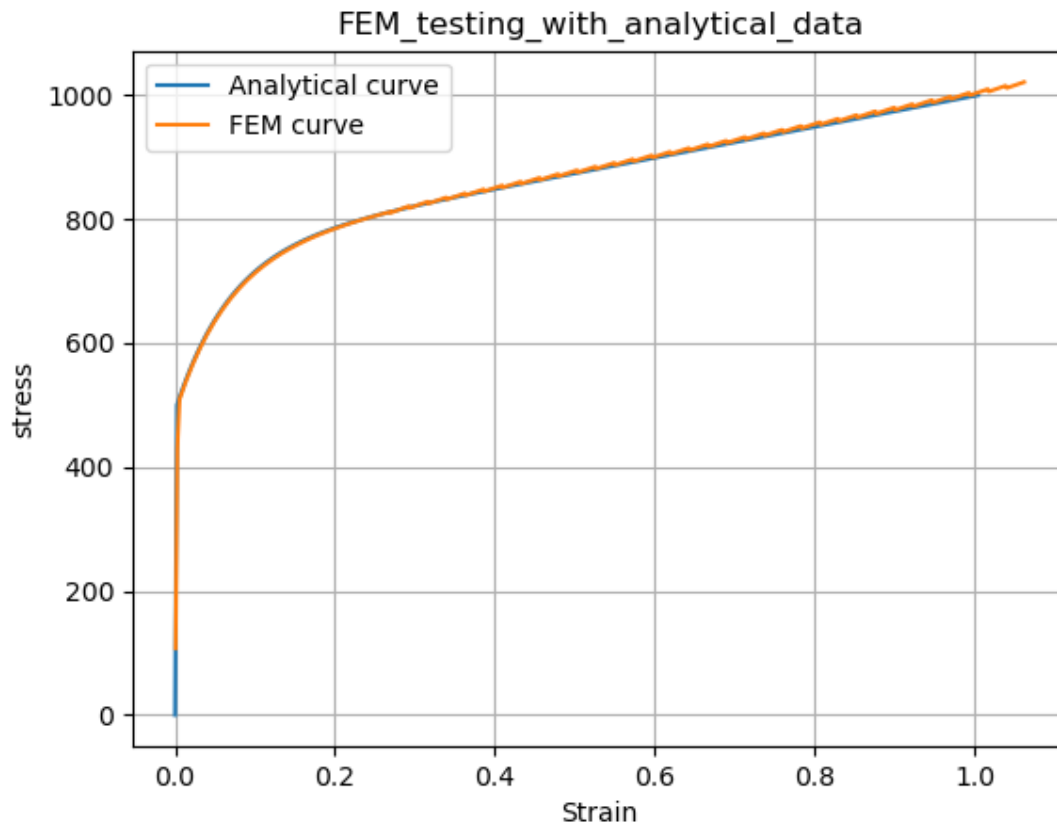


Fig: 12 experimental and FEM stress-strain curve

The analytical data and FEM simulated data matches perfectly. So, the difficult part of the FEM program is to code for non-linear hardening behavior. In the FEM von-mises plasticity model with isotropic non-linear hardening is used. Isotropic non-linear hardening can be solved by two laws they are voce hardening law(exponential law) and swift law(power-law). I used voce hardening law in the FEM program, so I am going to do the test with swift law which has to produce nearly the same data as FEM.

Swift law:

$$\sigma_y = k(\varepsilon_y + \varepsilon^{pl})^n$$

Where: k : strength co-efficient of the material

ε_y : strain at the yield point

ε^{pl} : plastic strain

n : strain hardening exponent of the material

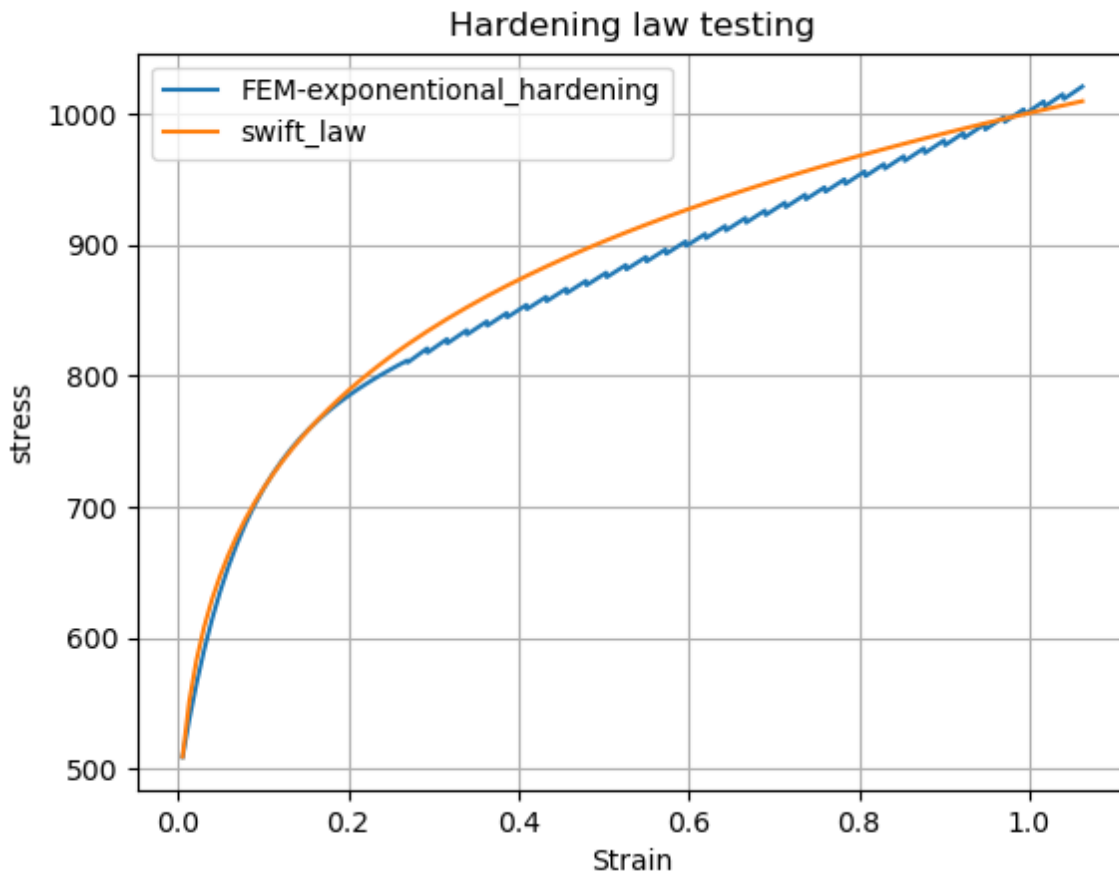


Fig: 13 FEM hardening and swift hardening

FEM simulated hardening and swift hardening are almost similar. The way of approach on these models differs so we encountered some wiggle in the curve.

To justify, the FEM curve fits with the analytical solution, voce hardening law and swift hardening law gives almost the same results. So, FEM program is valid.

Test case: 3

I used Levenberg-Marquardt algorithm to optimize the parameters of the material so I am checking with open-source python package called SciPy, which also generates the same fit as Levenberg-Marquardt algorithm. SciPy contains library for optimization, linear algebra, integration, interpolation, FFT, etc.

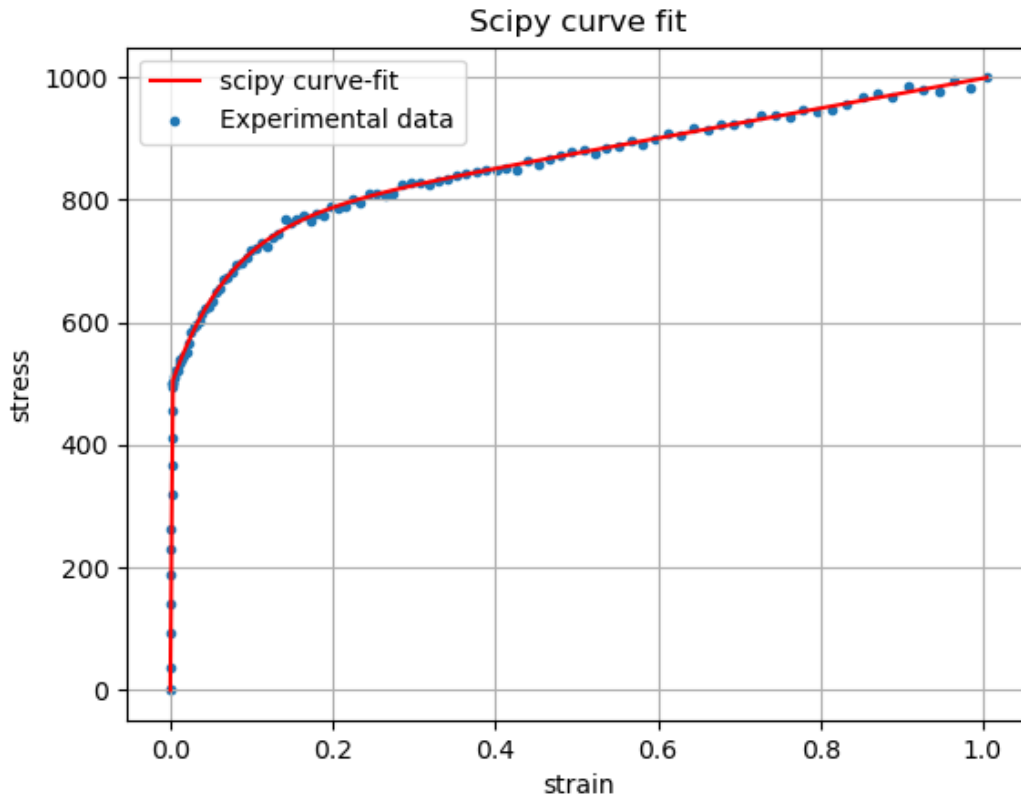


Fig: 14 SciPy curve fit with experimental data

SciPy optimized the parameters for the experimental data and the curve fits with the data perfectly. The below table compares the optimized parameters generated by SciPy and Levenberg-Marquardt algorithm.

Parameters	SciPy (MPa)	Levenberg-Marquardt (MPa)
Young's modulus(E)	215061.81	212742.99
Yield stress(σ_y)	492.4212	501.338
Linear hardening modulus(h)	243.8161	241.325
Nonlinear hardening modulus(σ_2)	261.8141	251.029
Hardening exponent(η)	14.06926	16.3

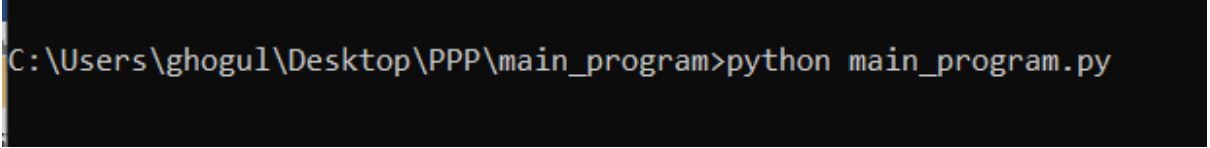
Table: 4 comparisons of SciPy optimization with Levenberg-Marquardt

From the above table, we can clearly see that the Levenberg-Marquardt algorithm optimized the parameters correctly.

5. Manual:

Just go to the folder “**main_program**” which contains all the necessary files to run the program.

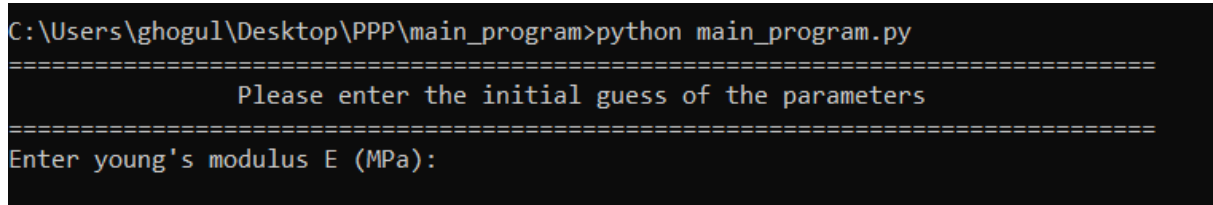
1. Open the terminal and give the correct path of the folder.
2. To run the program, type ”python” or “python3” based on your python version, then type “**main_program.py**”. if you run this program file it will automatically call other program files needed for the program.



```
C:\Users\ghogul\Desktop\PPP\main_program>python main_program.py
```

Fig: 15 steps 2

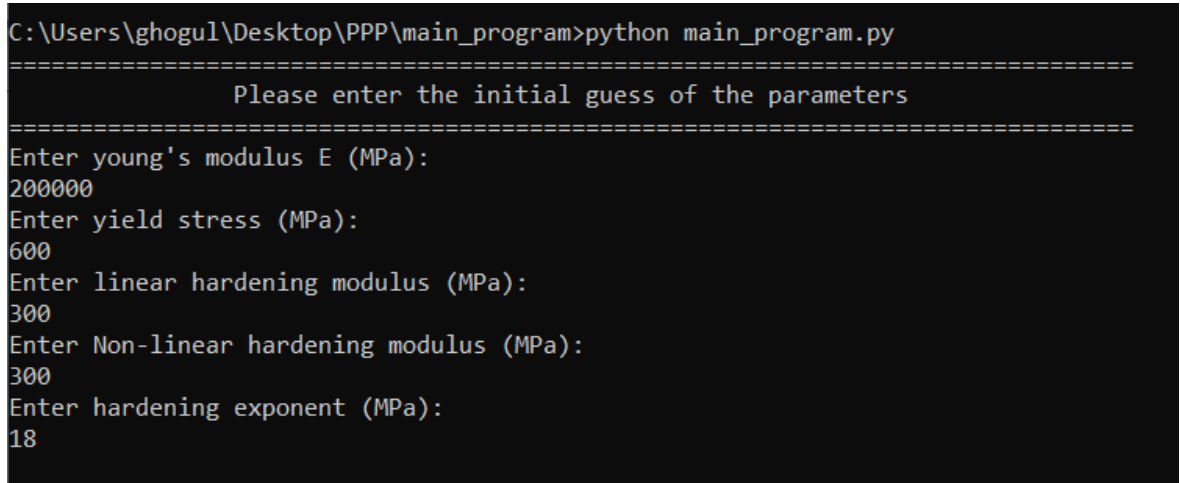
3. Then press “**enter**”.
4. Then the program will ask the user to give a guess as an initial parameters, for example, see fig: 16



```
C:\Users\ghogul\Desktop\PPP\main_program>python main_program.py
=====
Please enter the initial guess of the parameters
=====
Enter young's modulus E (MPa):
```

Fig: 16

5. You have to give five parameters to run the program successfully. See fig: 17 all the parameters are given, now programming is running.



```
C:\Users\ghogul\Desktop\PPP\main_program>python main_program.py
=====
Please enter the initial guess of the parameters
=====
Enter young's modulus E (MPa):
200000
Enter yield stress (MPa):
600
Enter linear hardening modulus (MPa):
300
Enter Non-linear hardening modulus (MPa):
300
Enter hardening exponent (MPa):
18
```

Fig: 17 user input

6. The output of the program will save in the “**main_program.txt**”. please open the text file for the detailed information. And all other related graphs are saved with the appropriate name.

Remarks:

As I mentioned in the previous section, the initial guess should not deviate much over than the experimental data. Try to guess the parameters which are in the trust region.

Error messages:

In case, your guess is bad or its too larger error message will pop-up in the terminal. See fig: 18

```
C:\Users\ghogul\Desktop\PPP\main_program>python main_program.py
=====
Please enter the initial guess of the parameters
=====
Enter young's modulus E (MPa):
200000
Enter yield stress (MPa):
900
Enter linear hardening modulus (MPa):
600
Enter Non-linear hardening modulus (MPa):
600
Enter hardening exponent (MPa):
20
!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!
Guess is too large !!!
The parameters are not in the trust region
Please guess the parameters which is close to the experimental data.
!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!
```

Fig: 18 Error message

I would suggest you give the values

Parameters	Suggested values
Young's modulus(E)	100000-300000
Yield stress(σ_y)	25-800
Linear hardening modulus(h)	25-550
Nonlinear hardening modulus(σ_2)	25-550
Hardening exponent(η)	2-20

Table: 5 suggested values for input

Literature:

1. O.C. Zienkiewicz, R.L. Taylor & J.Z. Zhu. The finite element method its basis & fundamentals, 6th edition, 2005
2. Prof. Björn Kiefer. Lecture “Plasticity”. Technische Universität Bergakademie Freiberg.
3. Prof. Björn Kiefer, Dr.-Ing. Martin Abendroth. lecture “Parameter Identification in Nonlinear Solid Mechanics”. Technische Universität Bergakademie Freiberg.

Git log:

Commits on Mar 30, 2020
code documentation is done  ghogul committed 6 days ago
code documentation is done  ghogul committed 6 days ago
code documentation is done  ghogul committed 6 days ago
code documentation is done  ghogul committed 6 days ago
code documentation is done  ghogul committed 6 days ago
Commits on Mar 22, 2020
algorithm testing, for polynomial its working perfectly fine  ghogul committed 13 days ago
Commits on Mar 9, 2020
LM is working fine but accuracy is missing have to increase accuracy  ghogul committed 26 days ago
now we can compare the simulation and experimental data  ghogul committed 26 days ago
Commits on Mar 8, 2020
checking with swift law  ghogul committed 27 days ago
everything is working perfectly fine  ghogul committed 27 days ago
Commits on Mar 7, 2020
looking for corrector parameters  ghogul committed 28 days ago

 Commits on Mar 6, 2020

converged properly

 ghogul committed 29 days ago

fully funcationable

 ghogul committed 29 days ago

nonlinear hardening also works perfectly.

 ghogul committed on Mar 6

 Commits on Mar 5, 2020

stress strain curve is generated

 ghogul committed on Mar 5

plasticity is converged and expected results came

 ghogul committed on Mar 5

 Commits on Mar 4, 2020

everything is converged but plasticity takes 20 iterations

 ghogul committed on Mar 4

 Commits on Mar 2, 2020

isotropic hardening in plasticity trial stress and corrected stress h... 

 ghogul committed on Mar 2

hardening is added

 ghogul committed on Mar 2

 Commits on Feb 24, 2020

elastic part is converged have to check for plasticity

 ghogul committed on Feb 24

changes in structural level

 ghogul committed on Feb 24

there is some changes in structural level check another file called s... 

 ghogul committed on Feb 24

 Commits on Feb 20, 2020

almost converged but not fully it clearly seems only a small mistake ... 

 **ghogul** committed on Feb 20

 Commits on Feb 18, 2020

have to change geometry and check for plastic strain

 **ghogul** committed on Feb 18

 Commits on Feb 17, 2020

only check for convergence no other changes have to decide displaceme... 

 **ghogul** committed on Feb 17

internal matrix is corrected, this also should be in element routine ... 

 **ghogul** committed on Feb 17

 Commits on Feb 16, 2020

some problem with internal force matrix and G reduced matrix the conv... 

 **ghogul** committed on Feb 16

difficulties in debugging the convergence only a small problem in Gma... 

 **ghogul** committed on Feb 16

 Commits on Feb 15, 2020

almost correected the convergence but not converged

 **ghogul** committed on Feb 15

 Commits on Feb 12, 2020

elasitic part success

 **ghogul** committed on Feb 12

checking convergence

 **ghogul** committed on Feb 12

 Commits on Feb 7, 2020

checking for convergence but no convergnce yet

 **ghogul** committed on Feb 7

Commits on Feb 4, 2020

global K is checked its not singular for numerical ...

 **ghogul** committed on Feb 4

Commits on Feb 3, 2020

checking gauss point 2

 **ghogul** committed on Feb 3

Commits on Feb 2, 2020

no changes ...

 **ghogul** committed on Feb 2

global stiffness matrix is not singlar with 2 gaus ...

 **ghogul** committed on Feb 2

gauss point 2 is checked but global K is not singl

 **ghogul** committed on Feb 2

Commits on Feb 1, 2020

gauss point 1 is used

 **ghogul** committed on Feb 1

gauss point 2 is used

 **ghogul** committed on Feb 1

Commits on Jan 31, 2020

CREATED FUNCTIONS

 **ghogul** committed on Jan 31

all elements coordinates are in function now ...

 **ghogul** committed on Jan 31

Commits on Jan 30, 2020

NO CHANGES

 **ghogul** committed on Jan 30

GLOBAL STIFFNESS MATRIX IS CORRECTED

 **ahoaul** committed on Jan 30